

我个人比较忧虑这个做法, 我认为它为了伶俐巧妙的修补而牺牲了清晰性。然而将“*copying* 动作”从函数本体内移至“函数参数构造阶段”却可令编译器有时生成更高效的代码。

请记住

- 确保当对象自我赋值时 `operator=` 有良好行为。其中技术包括比较“来源对象”和“目标对象”的地址、精心周到的语句顺序、以及 `copy-and-swap`。
- 确定任何函数如果操作一个以上的对象, 而其中多个对象是同一个对象时, 其行为仍然正确。

条款 12: 复制对象时勿忘其每一个成分

Copy all parts of an object.

设计良好之面向对象系统 (OO-systems) 会将对象的内部封装起来, 只留两个函数负责对象拷贝 (复制), 那便是带着適切名称的 *copy* 构造函数和 *copy assignment* 操作符, 我称它们为 *copying* 函数。条款 5 观察到编译器会在必要时候为我们的 `classes` 创建 *copying* 函数, 并说明这些“编译器生成版”的行为: 将被拷对象的所有成员变量都做一份拷贝。

如果你声明自己的 *copying* 函数, 意思就是告诉编译器你并不喜欢缺省实现中的某些行为。编译器仿佛被冒犯似的, 会以一种奇怪的方式回敬: 当你的实现代码几乎必然出错时却不告诉你。

考虑一个 `class` 用来表现顾客, 其中手工写出 (而非由编译器创建) *copying* 函数, 使得外界对它们的调用会被志记 (logged) 下来:

```
void logCall(const std::string& funcName);    //制造一个 log entry
class Customer {
public:
    ...
    Customer(const Customer& rhs);
    Customer& operator=(const Customer& rhs);
    ...
private:
    std::string name;
};
```

```

Customer::Customer(const Customer& rhs)
    : name(rhs.name)                //复制 rhs 的数据
{
    logCall("Customer copy constructor");
}

Customer& Customer::operator=(const Customer& rhs)
{
    logCall("Customer copy assignment operator");
    name = rhs.name;                //复制 rhs 的数据
    return *this;                   //见条款 10
}

```

这里的每一件事情看起来都很好，而实际上每件事情也的确都好，直到另一个成员变量加入战局：

```

class Date { ... };                //日期
class Customer {
public:
    ...                            //同前
private:
    std::string name;
    Date lastTransaction;
};

```

这时候既有的 *copying* 函数执行的是局部拷贝（*partial copy*）：它们的确复制了顾客的 *name*，但没有复制新添加的 *lastTransaction*。大多数编译器对此不出任何怨言——即使在最高警告级别中（见条款 53）。这是编译器对“你自己写出 *copying* 函数”的复仇行为：既然你拒绝它们为你写出 *copying* 函数，如果你的代码不完全，它们也不告诉你。结论很明显：如果你为 *class* 添加一个成员变量，你必须同时修改 *copying* 函数。（你也需要修改 *class* 的所有构造函数（见条款 4 和条款 45）以及任何非标准形式的 *operator=*（条款 10 有个例子）。如果你忘记，编译器不太可能提醒你。）

一旦发生继承，可能会造成此一主题最暗中肆虐的一个潜藏危机。试考虑：

```

class PriorityCustomer: public Customer {    //一个 derived class
public:
    ...
    PriorityCustomer(const PriorityCustomer& rhs);
    PriorityCustomer& operator=(const PriorityCustomer& rhs);
    ...
private:
    int priority;
};

```

```

PriorityCustomer::PriorityCustomer(const PriorityCustomer& rhs)
    : priority(rhs.priority)
{
    logCall("PriorityCustomer copy constructor");
}

PriorityCustomer&
PriorityCustomer::operator=(const PriorityCustomer& rhs)
{
    logCall("PriorityCustomer copy assignment operator");
    priority = rhs.priority;
    return *this;
}

```

PriorityCustomer 的 *copying* 函数看起来好像复制了 PriorityCustomer 内的每一样东西, 但是请再看一眼。是的, 它们复制了 PriorityCustomer 声明的成员变量, 但每个 PriorityCustomer 还内含它所继承的 Customer 成员变量复件 (副本), 而那些成员变量却未被复制。PriorityCustomer 的 *copy* 构造函数并没有指定实参传给其 base class 构造函数 (也就是说它在它的成员初值列 (member initialization list) 中没有提到 Customer), 因此 PriorityCustomer 对象的 Customer 成分会被不带实参之 Customer 构造函数 (即 *default* 构造函数——必定有一个否则无法通过编译) 初始化。*default* 构造函数将针对 name 和 lastTransaction 执行缺省的初始化动作。

以上事态在 PriorityCustomer 的 *copy assignment* 操作符身上只有轻微不同。它不曾企图修改其 base class 的成员变量, 所以那些成员变量保持不变。

任何时候只要你承担起“为 derived class 撰写 *copying* 函数”的重责大任, 必须很小心地也复制其 base class 成分。那些成分往往是 *private* (见条款 22), 所以你不能直接访问它们, 你应该让 derived class 的 *copying* 函数调用相应的 base class 函数:

```

PriorityCustomer::PriorityCustomer(const PriorityCustomer& rhs)
    : Customer(rhs), //调用base class 的 copy 构造函数
      priority(rhs.priority)
{
    logCall("PriorityCustomer copy constructor");
}

PriorityCustomer&
PriorityCustomer::operator=(const PriorityCustomer& rhs)
{
    logCall("PriorityCustomer copy assignment operator");
    Customer::operator=(rhs); //对base class 成分进行赋值动作
    priority = rhs.priority;
    return *this;
}

```

本条款题目所说的“复制每一个成分”现在应该很清楚了。当你编写一个 *copying* 函数，请确保 (1) 复制所有 local 成员变量，(2) 调用所有 base classes 内的适当的 *copying* 函数。

这两个 *copying* 函数往往有近似相同的实现本体，这可能会诱使你让某个函数调用另一个函数以避免代码重复。这样精益求精的态度值得赞赏，但是令某个 *copying* 函数调用另一个 *copying* 函数却无法让你达到你想要的目标。

令 *copy assignment* 操作符调用 *copy* 构造函数是不合理的，因为这就像试图构造一个已经存在的对象。这件事如此荒谬，乃至根本没有相关语法。是有一些看似如你所愿的语法，但其实不是；也的确有些语法背后真正做了它，但它们在某些情况下会造成你的对象败坏，所以我不打算将那些语法呈现给你看。单纯地接受这个叙述吧：你不该令 *copy assignment* 操作符调用 *copy* 构造函数。

反方向——令 *copy* 构造函数调用 *copy assignment* 操作符——同样无意义。构造函数用来初始化新对象，而 *assignment* 操作符只施行于已初始化对象身上。对一个尚未构造好的对象赋值，就像在一个尚未初始化的对象身上做“只对已初始化对象才有意义”的事一样。无聊嘛！别尝试。

如果你发现你的 *copy* 构造函数和 *copy assignment* 操作符有相近的代码，消除重复代码的做法是，建立一个新的成员函数给两者调用。这样的函数往往是 *private* 而且常被命名为 *init*。这个策略可以安全消除 *copy* 构造函数和 *copy assignment* 操作符之间的代码重复。

请记住

- *Copying* 函数应该确保复制“对象内的所有成员变量”及“所有 base class 成分”。
- 不要尝试以某个 *copying* 函数实现另一个 *copying* 函数。应该将共同机能放进第三个函数中，并由两个 *copying* 函数共同调用。